

Parte 2: Introducción a Wireshark

Redes de Computadoras - CC308

José Antonio Mérida Castejón - 201105

13 de julio de 2026

Introducción

En la primera parte de este laboratorio se replicaron esquemas de comunicación previos a las redes modernas (código Morse, código Baudot y un protocolo propio de conmutación de mensajes), lo cual permitió evidenciar de forma práctica los problemas de fiabilidad, ergonomía y enrutamiento que enfrentaban dichos esquemas. En esta segunda parte se aborda una herramienta central para el estudio de las redes de computadoras actuales: Wireshark, un analizador de protocolos que permite capturar y examinar el tráfico que circula por una interfaz de red en tiempo real.

El objetivo de esta parte es familiarizarse con el entorno de Wireshark, personalizando perfiles, layout y filtros, para luego realizar una captura de paquetes sobre la interfaz de red activa de la computadora y analizar una solicitud HTTP real, identificando las versiones del protocolo utilizadas, los encabezados relevantes de la solicitud y de la respuesta, y discutiendo consideraciones de rendimiento y seguridad relacionadas con el uso de esta herramienta.

Personalización del Entorno

Configuración de perfil

Se creó un nuevo perfil en Wireshark para personalizar la captura sin afectar la configuración por defecto.

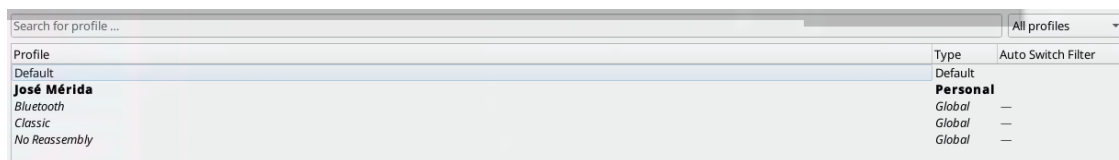


Figura 1: Creación del perfil personalizado en Wireshark.

Layout, Filtro y Botón de Filtro

Se modificó el layout de la interfaz, agregando un botón de filtro y una regla de coloreo personalizada.

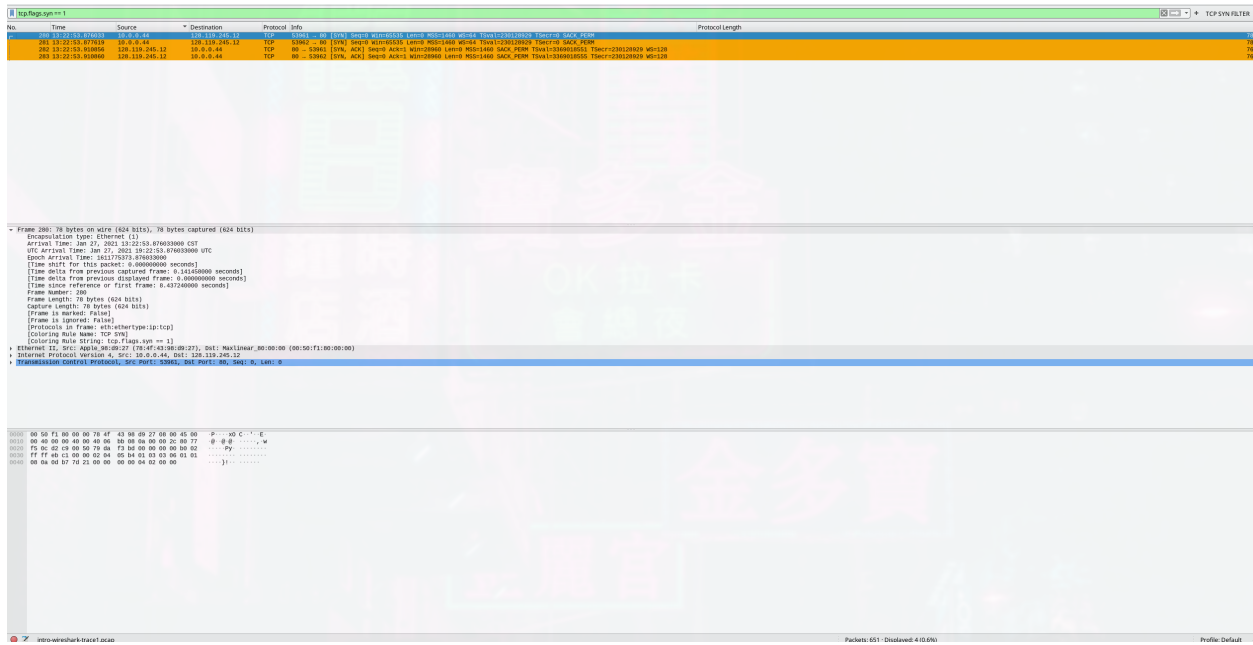


Figura 2: Cambio de layout, botón agregado y regla de coloreo.

Name	Filter
☒ TCP SYN	tcp.flags.syn == 1
☒ Bad TCP	tcp.analysis.flags && !tcp.analysis.window_update && !tcp.analysis.keep_alive && !tcp.analysis.keep_alive_ack
☒ HSRP State Change	hsrp.state != 8 && hsrp.state != 16
☒ Spanning Tree Topology Change	stp.type == 0x80
☒ OSPF State Change	ospf.msg != 1
☒ ICMP errors	icmp.type in { 3, 5, 11 } !icmpv6.type in { 1, 4 }
☒ ARP	arp
☒ ICMP	icmp icmpv6
☒ TCP RST	tcp.flags.reset eq 1
☒ SCTP ABORT	sctp.chunk_type eq ABORT
☒ IPv4 TTL low or unexpected	(ip.dst != 224.0.0.0/4 && ip.ttl < 5 && !ospf !eigrp !bgp tcp.port == 179) (ip.dst == 224.0.0.0/24 && ip.dst != 224.0.0.251 && ip.ttl != 1 && !vrrp !carp !eigrp !rip !ospf !bgp !tcp.port == 179) (ip.v6.dst == ff00::/8 && ip.v6.hlim < 5 && !ospf !bgp !tcp.port == 179) (ip.v6.dst == ff00::/8 && ip.v6.hlim not in { 1, 64, 255 })
☒ IPv6 hop limit low or unexpected	(ip.v6.dst != ff00::/8 && ip.v6.hlim < 5 && !ospf !bgp !tcp.port == 179) (ip.v6.dst == ff00::/8 && ip.v6.hlim not in { 1, 64, 255 })
☒ Checksum Errors	eth.fcs.status == "Bad" ip.checksum.status == "Bad" tcp.checksum.status == "Bad" udp.checksum.status == "Bad" sctp.checksum.status == "Bad" mstp.checksum.status == "Bad"
☒ SMB	smb nbss nbns netbios
☒ HTTP	http tcp.port == 80 http2
☒ DCE/RPC	dcerpc
☒ Routing	hsrp eigrp ospf bgp cdp vrrp carp gvrp igmp ismp
☒ TCP SYN/FIN	tcp.flags & 0x02 tcp.flags.fin == 1
☒ TCP	tcp
☒ UDP	udp
☒ Broadcast	eth[0] & 1
☒ System Event	systemd_journal sysdig

Figura 3: Regla de coloreo personalizada agregada en Wireshark.

Interfaces Virtuales

Se removieron las interfaces virtuales o que no se encuentran en uso dentro del sistema, esto también se verifica por medio del comando `ifconfig` ejecutado en una sección posterior del laboratorio.

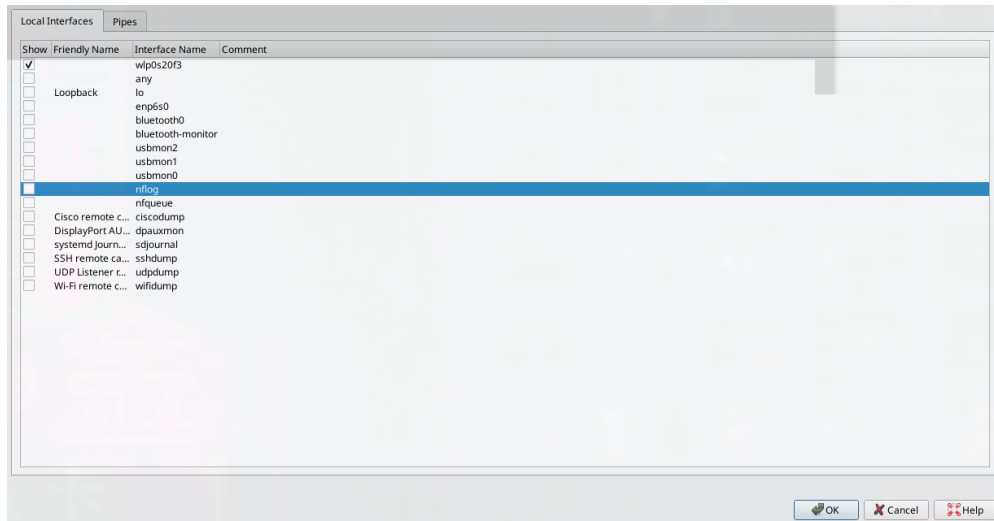


Figura 4: Listado de interfaces sin las interfaces virtuales.

Configuración de la Captura de Paquetes

```

) ifconfig
enp6s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 08:bf:b8:a0:11:50 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device memory 0x82f00000-82ffffff

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1141 bytes 137838 (134.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1141 bytes 137838 (134.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.17 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::aa69:feb:4926:90d1 prefixlen 64 scopeid 0x20<link>
    inet6 2800:98:1124:aa7::1 prefixlen 128 scopeid 0x0<global>
    inet6 2800:98:1124:aa7:ff97:6b9e:b3c6:6e5a prefixlen 64 scopeid 0x0<global>
    ether 0e:62:e7:72:da:65 txqueuelen 1000 (Ethernet)
    RX packets 34769 bytes 34577387 (32.9 MiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 15481 bytes 7619843 (7.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Figura 5: Ejecución de comando ifconfig.

El comando `ifconfig` muestra todos los adaptadores de red disponibles en la computadora, al igual que sus direcciones IP (en caso de tener una asignada) e ID's de hardware. En este caso podemos ver tres dispositivos:

- **enp6s0**: Interfaz de ethernet, no tiene IP asignada y no ha mandado o recibido ningún paquete. Esto es consistente con el hecho que no está conectado ningún cable de ethernet a la computadora al momento de ejecutar el comando.
- **lo**: Tiene asignada la IP `127.0.0.1`, es una interfaz virtual utilizada por el sistema operativo. Este es el localhost.

- **wlp0s20f3**: Interfaz de internet inalámbrica, se puede ver la IP asignada 192.168.1.17 al igual que los paquetes transmitidos (TX) y paquetes recibidos (RX) desde el momento de arranque. Esta es la interfaz de red activa del sistema.

Utilizando el dispositivo **wlp0s20f3** se realizó la captura con la siguiente configuración:

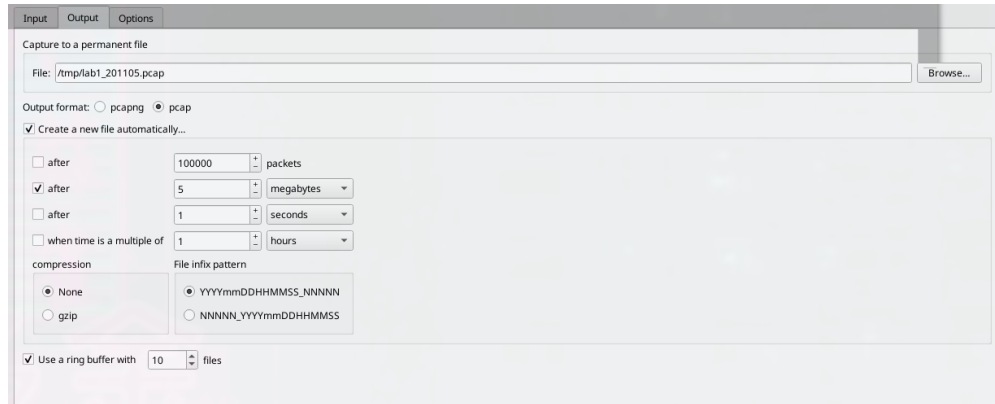


Figura 6: Configuración de las opciones de salida de la captura en Wireshark.

Los archivos resultantes se encuentran dentro del repositorio de la entrega, dentro del directorio **files**.

Análisis de Paquetes

Para el análisis se filtró el tráfico capturado utilizando el filtro **http**, aislando así únicamente las solicitudes y respuestas del protocolo HTTP generadas al cargar la página de prueba del laboratorio (**gaia.cs.umass.edu**).

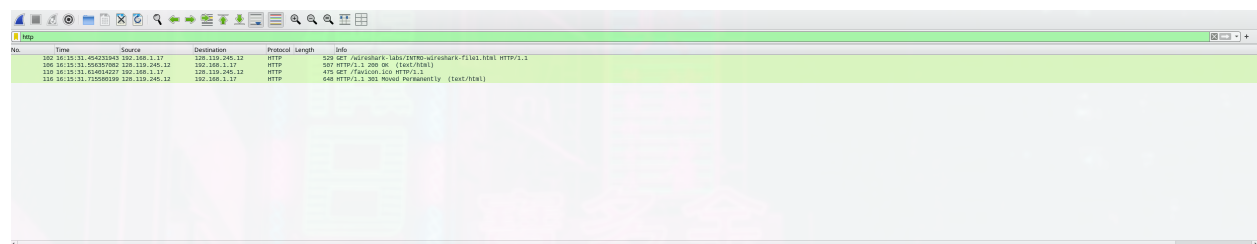


Figura 7: Solicitudes y respuestas HTTP capturadas al filtrar por **http**.

Preguntas

- ¿Qué versión de HTTP está ejecutando su navegador?
HTTP/1.1, evidenciado por la solicitud GET inicial hacia la página.
- ¿Qué versión de HTTP está ejecutando el servidor?
HTTP/1.1, evidenciado por la respuesta 200 OK del servidor.

- c. ¿Qué lenguajes (si aplica) indica el navegador que acepta a el servidor?

```
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
\r\n
[Response in frame: 106]
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]
```

Figura 8: Encabezados de la solicitud GET, incluyendo Accept-Language.

en-US, en. Inglés Americano e Inglés.

- d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

```
Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
    Date: Mon, 13 Jul 2026 22:15:31 GMT\r\n
    Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.12 Perl/v5.32.1\r\n
    Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n
    ETag: "51-64231b6715777"\r\n
    Accept-Ranges: bytes\r\n
    Content-Length: 81\r\n
    [Content length: 81]
    Keep-Alive: timeout=5, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: text/html; charset=UTF-8\r\n
    \r\n
    [Request in frame: 102]
```

Figura 9: Encabezados de la respuesta HTTP, incluyendo Content-Length.

81 bytes.

- e. En el caso que haya un problema de rendimiento mientras se carga la página, ¿en qué elementos de la red convendría escuchar los paquetes?
- f. ¿Es conveniente instalar Wireshark en el servidor? Justifique

Discusión General

Conclusiones

Referencias

- [1] Wireshark Foundation, *Wireshark User's Guide: Chapter 7. Analyzing Protocols*. [En línea]. Disponible en: https://www.wireshark.org/docs/wsug_html_chunked/. [Accedido: 13-jul-2026].
- [2] SANS Institute, *Packet Capture Best Practices: Security and Performance Considerations*. [En línea]. Disponible en: <https://www.sans.org/white-papers/>. [Accedido: 13-jul-2026].
- [3] GeeksforGeeks, *ifconfig command in Linux with Examples*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/linux-unix/ifconfig-command-in-linux-with-examples/>. [Accedido: 13-jul-2026].
- [4] Stack Overflow, *How to check length of protocols*. [En línea]. Disponible en: <https://stackoverflow.com/questions/24197371/how-to-check-length-of-protocols>. [Accedido: 13-jul-2026].